PathFinder — API Documentation

Base URL

- **Local Development**: `http://localhost:5000`
- **Production**: `https://pathfinder-backend-j111.onrender.com`

Authentication

All protected endpoints require a JWT token in the Authorization header:

```
Authorization: Bearer <jwt-token>
```

---

1. Authentication Routes (`/api/auth`)

Rate Limited: 5 requests per 15 minutes per IP.

POST /api/auth/register

Create a new user account.

Request Body:

```
{
  "email": "student@example.com",
  "password": "SecurePass123",
  "role": "STUDENT",
  "firstName": "Aarav",
  "lastName": "Sharma"
}
```

For organizations, replace firstName/lastName with `"companyName": "TechCorp"`.

Success Response (201):

```
{
  "token": "eyJhbGciOiJIUzI1...",
  "user": { "id": "uuid", "email": "student@example.com", "role": "STUDENT" }
}
```

Error Responses:

- 400: `{ "error": "Email already exists" }`
- 500: `{ "error": "Registration failed" }`

POST /api/auth/login

Authenticate an existing user.

Request Body:

```
{ "email": "student@example.com", "password": "SecurePass123" }
```

Success Response (200):

```
{
  "token": "eyJhbGciOiJIUzI1...",
  "user": { "id": "uuid", "email": "student@example.com", "role": "STUDENT" }
}
```

Error Responses:

- 400: `{ "error": "Invalid credentials" }`

POST /api/auth/forgot-password

Request a password reset email.

Request Body:

```
{ "email": "student@example.com" }
```

Response (200):

```
{ "message": "If an account exists, a reset link was sent." }
```

Note: Always returns success to prevent email enumeration.

POST /api/auth/reset-password

Reset password using token from email.

Request Body:

```
{ "token": "reset-token-string", "newPassword": "NewSecurePass456" }
```

Success Response (200):

```
{ "message": "Password reset successful" }
```

---

2. Student Routes (`/api/student`) — Auth Required (STUDENT)

GET /api/student/profile

Retrieve the authenticated student's full profile with skills, certifications, portfolio links, and documents.

PUT /api/student/profile

Update student profile fields (firstName, lastName, bio, major, university, graduationYear).

PUT /api/student/skills

Replace the student's entire skills list.
Request Body: `{ "skills": ["React", "Node.js", "Python"] }`

POST /api/student/certifications

Add a new certification.
Request Body: `{ "name": "AWS Certified", "issuer": "Amazon", "dateIssued": "2026-01" }`

DELETE /api/student/certifications/:id

Remove a certification by ID.

POST /api/student/portfolio-links

Add a portfolio link.
Request Body: `{ "title": "GitHub", "url": "https://github.com/username" }`

DELETE /api/student/portfolio-links/:id

Remove a portfolio link by ID.

---

3. Organization Routes (`/api/org`) — Auth Required (ORGANIZATION)

GET /api/org/profile

Retrieve the organization's profile.

PUT /api/org/profile

Update organization profile (companyName, description, website, industry, logoUrl).

---

4. Opportunity Routes (`/api/opportunities`)

GET /api/opportunities

List all published opportunities. Supports query parameters for filtering.
Query Params: `?category=INTERNSHIP&search=react&location=Remote`

GET /api/opportunities/:id

Get a single opportunity by ID with organization details.

POST /api/opportunities — Auth Required (ORGANIZATION)

Create a new opportunity.
Request Body:

```
{
  "title": "Summer Internship 2026",
  "description": "Join our engineering team...",
  "category": "INTERNSHIP",
  "deadline": "2026-08-01T00:00:00Z",
  "eligibility": "CS students, 3rd year+",
  "location": "Remote",
  "stipend": "$2000/month",
  "duration": "3 months"
}
```

PUT /api/opportunities/:id — Auth Required (ORGANIZATION)

Update an existing opportunity (only if owned by the authenticated organization).

DELETE /api/opportunities/:id — Auth Required (ORGANIZATION)

Delete an opportunity (only if owned by the authenticated organization).

---

5. Application Routes (`/api/applications`)

POST /api/applications — Auth Required (STUDENT)

Submit an application. Triggers AI match scoring in background.
Request Body: `{ "opportunityId": "uuid", "coverLetter": "Dear Hiring Manager..." }`

GET /api/applications/student — Auth Required (STUDENT)

Get all applications for the authenticated student.
Query Params: `?status=PENDING`

PUT /api/applications/:id/withdraw — Auth Required (STUDENT)

Withdraw an application (cannot withdraw accepted applications).

GET /api/applications/:id/timeline — Auth Required

Get the event timeline for a specific application.

GET /api/applications/org/:opportunityId — Auth Required (ORGANIZATION)

Get all applications for a specific opportunity with student details and match scores.

PUT /api/applications/:id/status — Auth Required (ORGANIZATION)

Update a single application's status.
Request Body: `{ "status": "ACCEPTED", "notes": "Great candidate!" }`

PUT /api/applications/batch/status — Auth Required (ORGANIZATION)

Batch update multiple application statuses.
Request Body: `{ "applicationIds": ["uuid1", "uuid2"], "status": "REVIEWING" }`

---

6. AI Routes (`/api/ai`) — Auth Required

POST /api/ai/parse-resume (STUDENT)

Upload a PDF resume for AI-powered profile extraction. Accepts multipart form data with field name "resume".

POST /api/ai/generate-sop (STUDENT)

Generate a Statement of Purpose.
Request Body: `{ "opportunityId": "uuid", "additionalContext": "I have 2 years of experience..." }`

POST /api/ai/generate-cover-letter (STUDENT)

Generate a Cover Letter.

POST /api/ai/generate-essay (STUDENT)

Generate an Essay.
Request Body: `{ "opportunityId": "uuid", "essayPrompt": "Why technology matters", "additionalContext": "..." }`

POST /api/ai/generate-personal-statement (STUDENT)

Generate a Personal Statement.

GET /api/ai/profile-strength (STUDENT)

Get profile completeness score with checklist.

GET /api/ai/recommendations (STUDENT)

Get top 10 recommended opportunities based on student's skills.

GET /api/ai/match-score/:opportunityId (STUDENT)

Get match score for a specific opportunity.

GET /api/ai/candidate-summary/:applicationId (ORGANIZATION)

Get AI-generated candidate summary for an applicant.

GET /api/ai/history

Get the authenticated user's AI generation history.

---

7. Ticket Routes (`/api/tickets`) — Auth Required

POST /api/tickets

Create a new support ticket.
Request Body: `{ "subject": "Bug Report", "description": "Details...", "type": "BUG", "priority": "HIGH" }`

GET /api/tickets/mine

Get the authenticated user's tickets.

GET /api/tickets/admin (ADMIN only)

Get all tickets with creator details.

GET /api/tickets/:id

Get a specific ticket with all messages. Access restricted to creator and admins.

PUT /api/tickets/:id/status (ADMIN only)

Update ticket status.
Request Body: `{ "status": "RESOLVED" }`

POST /api/tickets/:id/messages

Add a message to a ticket. Access restricted to creator and admins.
Request Body: `{ "message": "Thanks for reporting this..." }`

---

8. Admin Routes (`/api/admin`) — Auth Required (ADMIN)

GET /api/admin/analytics

Platform-wide analytics.

GET /api/admin/analytics/export

Export analytics as CSV file download.

GET /api/admin/users

List users with pagination and search.
Query Params: `?page=1&limit=20&search=email&role=STUDENT`

PUT /api/admin/users/:id/role

Change a user's role.
Request Body: `{ "role": "ADMIN" }`

DELETE /api/admin/users/:id

Delete a user (cannot delete self).

GET /api/admin/opportunities

List all opportunities with search.

DELETE /api/admin/opportunities/:id

Delete any opportunity.

GET /api/admin/audit-log

Paginated audit log.

GET /api/admin/system-health

Server health check (uptime, memory, database counts, Node version).

---

9. Other Routes

GET /api/notifications — Auth Required

Get user's notifications.

PUT /api/notifications/read-all — Auth Required

Mark all notifications as read.

GET /api/search?q=keyword — Auth Required

Search opportunities by keyword.

GET /api/stats — Auth Required

Dashboard statistics.

POST /api/upload — Auth Required

Upload a file (multipart/form-data, field: "file"). Returns file URL.

GET /api/files/:filename

Retrieve uploaded file. Images are public; documents require auth.

End of API Documentation